BACK-END DEVELOPMENT

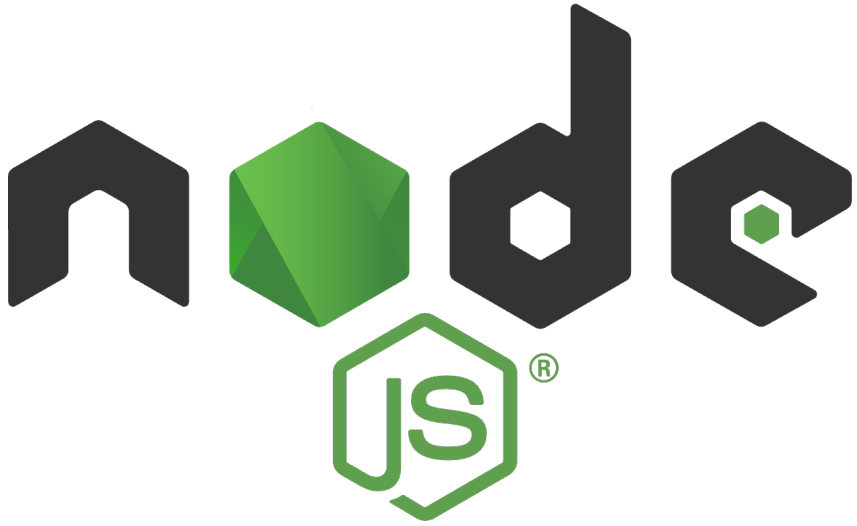
WEEK 2 – Native HTTP Server & Manual Routing

CADT IDT

After Finishing This Lecture:

- Create a basic HTTP server using Node.js
- Manually route HTTP requests using method + URL
- Extract query parameters and request body
- Understand how request and response objects work

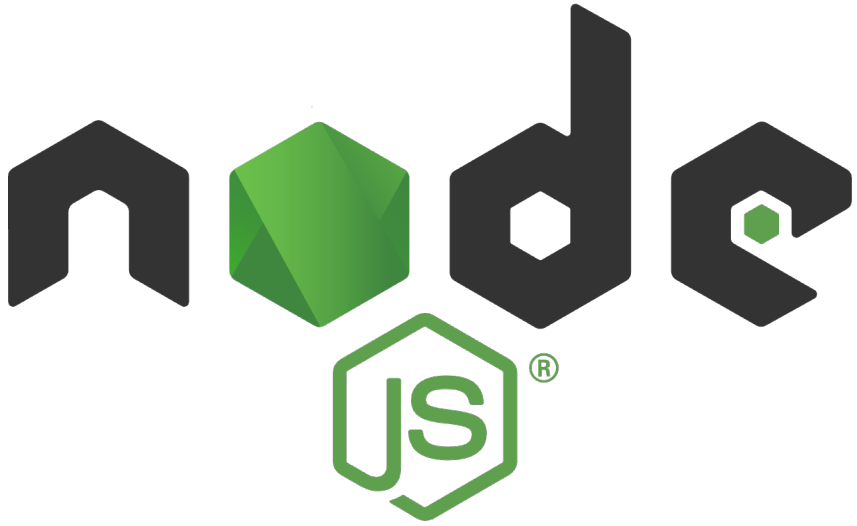
What is http in node.js



- Built-in module to create web servers.
- Provides tools to handle requests, responses, and network events.
- No need to install it. It's part of Node.js core.

```
1 const http = require('http');
```

What's Inside the http Module?



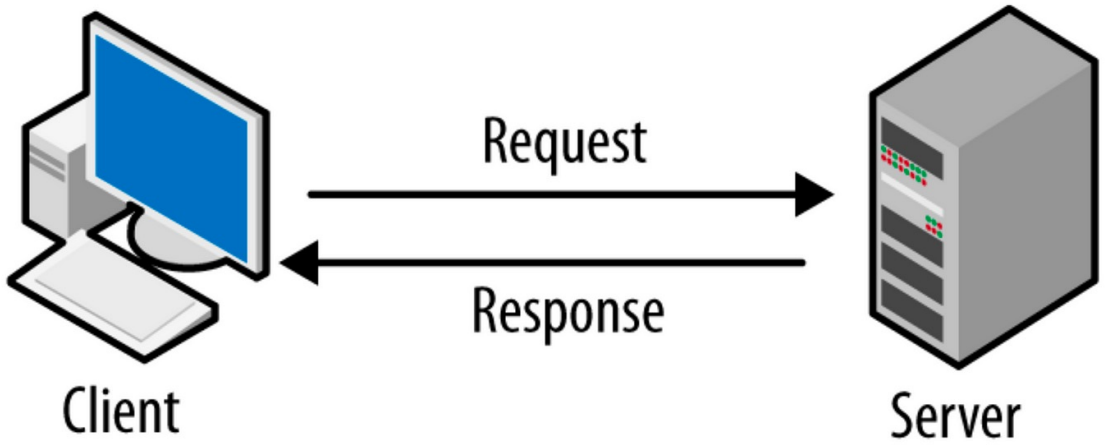
- http.createServer(): Creates a server instance.
- req (IncomingMessage): represents the client request.
- res (ServerResponse): represents the server response.

```
1 const server = http.createServer((req, res) => {
2   // Handle incoming requests here
3 });
4
```

Creating an HTTP Server

- **Activity**
- What happens if we don't call res.end()?
- Why is the callback in listen() useful?

```
1 const http = require('http');
2 const server = http.createServer((req, res) => {
3   res.end('Hello World!');
4 });
5 server.listen(3000, () => {
6   console.log('Server running on http://localhost:3000');
7 });
```



What is res.end()?

res.end() is a method on the Server Response object that **ends the response process**.

It tells the server, *"I'm done sending data to the client."*

Real-World Analogy

Think of req as **a letter someone mails to you**, with:

Their name, message (body), metadata (headers), and return address.

res is **your reply letter**, where:

You choose what to send back, what tone (status code), and how it’s packaged (headers).

Manual Routing

Routing means deciding **what code to run** based on:

HTTP Method: GET, POST, PUT, DELETE, etc.

URL Path: /, /about, /api/user, etc.

Manual Routing

- req.method: checks the **HTTP method** (GET, POST, etc.)
- req.url: checks the **requested path** (/, /about, etc.)
- You use basic if/else or switch to match routes.

```
1 const http = require('http');
2
3 const server = http.createServer((req, res) => {
4   const { method, url } = req;
5
6   if (method === 'GET' && url === '/') {
7     res.writeHead(200, { 'Content-Type': 'text/plain' });
8     res.end('Welcome to the homepage!');
9   } else if (method === 'GET' && url === '/about') {
10    res.writeHead(200, { 'Content-Type': 'text/plain' });
11    res.end('This is the About page.');
12  } else {
13    res.writeHead(404, { 'Content-Type': 'text/plain' });
14    res.end('404 Not Found');
15  }
16 });
17
18 server.listen(3000, () => {
19   console.log('Server listening on port 3000');
20 });
```

mini challenge

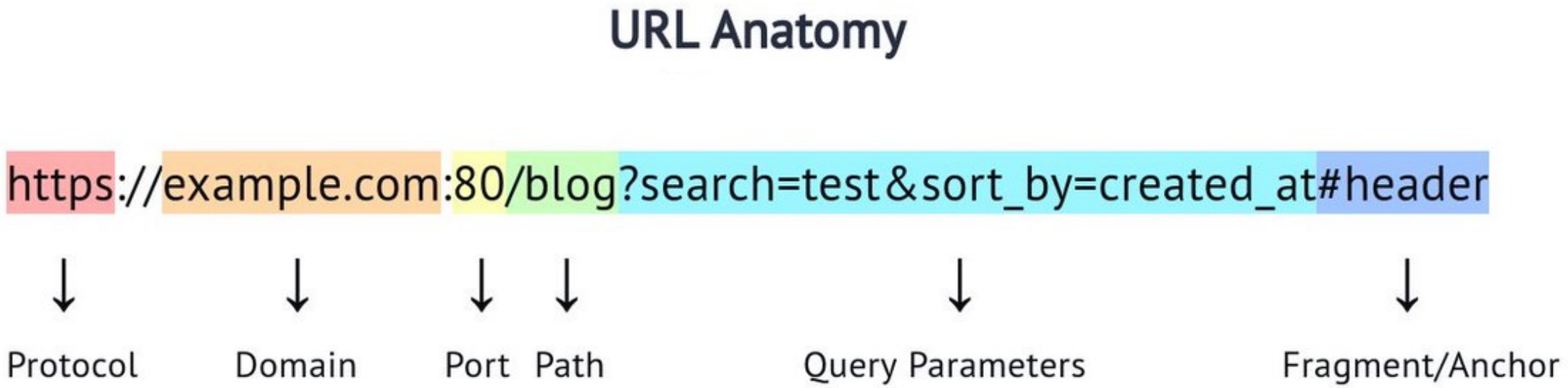
✏️ *Add a new route: GET /contact that responds with “Contact us at support@example.com.”*

Bonus: Use switch for Cleaner Routing

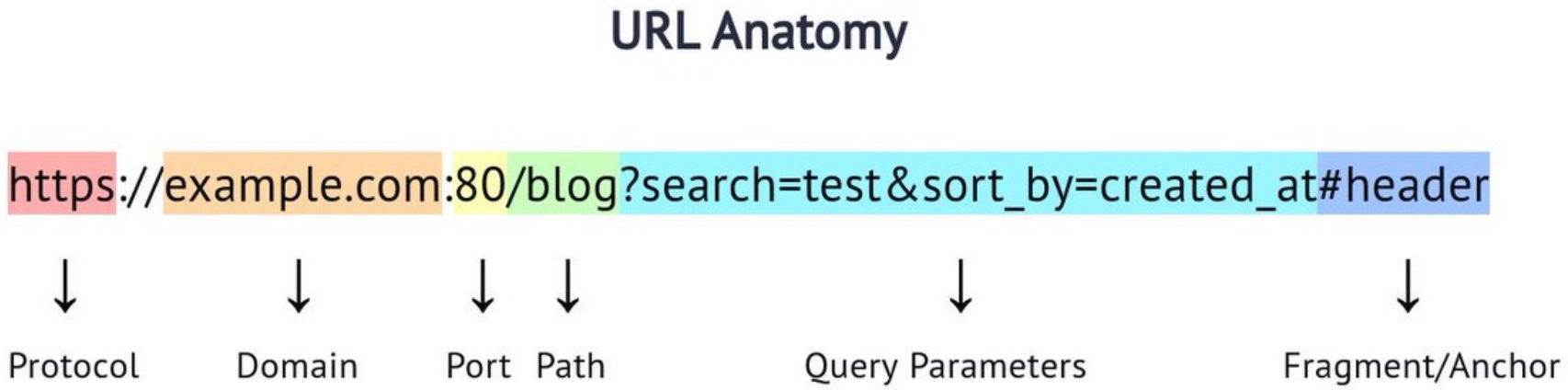
```
switch (true) {
  case method === 'GET' && url === '/':
    // handle home
    break;
  case method === 'GET' && url === '/about':
    // handle about
    break;
  default:
    // 404
}
```

Web Components

Uniform Resource Locator (URL)



How can we handle that URL in node's http module?




```
curl "http://localhost:8080/blog?search=sdfsdf&sort_by=sdfsd"
```

```
Method = GET
Path = /blog
Query:
   - search=test
   - sort_by=created_at
```

How can we handle that URL in node's http module?

```
1 const http = require('http');
2 const url = require('url');
3
4 const server = http.createServer((req, res) => {
5     if (req.method === 'GET' && req.url.includes('/blog')) {
6         const parsedUrl = url.parse(req.url, true);
7         const query = parsedUrl.query;
8         res.end(`Welcome to homepage search=${query.search} sort_by=${query.sort_by}`);
9     }
10 });
```

```
Method = GET
Path = /blog
Query:
    - search=test
    - sort_by=created_at
```

Property	Description
req.method	HTTP method like GET, POST, etc.
req.url	The requested path (can include query string)
req.headers	Object of key-value headers
req.body	🚫 Does NOT exist by default! You must read it manually using streams

Request and Response Objects

Key Properties of req (Request)

Property/Method	Description
res.writeHead()	Sets status code and headers
res.write()	Sends chunks of data
res.end()	Finalizes and sends the response
res.setHeader()	Sets headers individually

Request and Response Objects

Key Methods/Properties of res (Response)

Reading the Body (e.g., POST request)

Why Is the Body a Stream?

- req is a **readable stream**, and the body arrives in **chunks** (especially for large payloads).

- You must **collect chunks** and then process the whole body when the stream ends.

```
let body = '';
req.on('data', chunk => {
  body += chunk;
});

req.on('end', () => {
  console.log('Received body:', body);
  // Optionally parse JSON
  const parsed = JSON.parse(body);

  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ message: `Hello, ${parsed.name}` }));
});
```

How to POST request with body using CURL

```
curl -X POST https://example.com/api \
  -H "Content-Type: application/json" \
  -d '{"key1":"value1", "key2":"value2"}'
```

- **-X POST**: explicitly sets the HTTP method (optional if using -d)
- **-H "Content-Type**: application/json": tells the server you're sending JSON
- **-d**: sends the request body

Get Your Hand Dirty

- What happens if you send malformed JSON?
- What if you send data in URL-encoded format instead?

WHAT WE HAVE LEARNT

- Create a basic HTTP server using Node.js
- Manually route HTTP requests using method + URL
- Extract query parameters and request body
- Understand how request and response objects work